



Aplicació de Gestió Bancària

Pràctica de Disseny Orientat a Objectes (DOO)

Mòdul 5 - Entorns de Desenvolupament

Curs 2025-2026

Autor: Alumne

Data: Maig 2026

Índex

1. Anàlisi gramatical (Mètode Booch)
2. Justificació de classes i relacions
3. Codi Java amb JavaDoc (Part 2)
4. Diagrames UML (Part 2)
5. Patrons de disseny (Part 3)
6. Documentació JavaDoc generada (Part 3)

1. Anàlisi Gramatical - Mètode Booch

1.1 Classificació de paraules clau

S'ha aplicat el mètode de Grady Booch sobre l'enunciat original, classificant les paraules clau en:

- **Noms comuns** → **Classes** (Banc, Client, Compte, Sucursal...)
- **Verbs d'acció** → **Mètodes** (gestionar, ingressar, retirar...)
- **Verbs d'existència** → **Herència** ("hi ha dos tipus de comptes")
- **Verbs de possessió** → **Composició/Agregació** ("tenen", "poden tenir", "estan compostes")
- **Adjectius/Noms descriptius** → **Atributs** (DNI, nom, saldo, número...)

1.2 Classes candidates identificades

#	Classe	Tipus	Justificació
1	Persona	Abstracta	Agrupa atributs comuns de Client i Empleat
2	Client	Concreta	Persona que contracta productes bancaris
3	Empleat	Concreta	Persona que treballa en una sucursal
4	Sucursal	Concreta	Oficina bancària amb empleats
5	Producte	Abstracta	Generalització de tots els productes bancaris
6	Compte	Abstracta	Producte amb saldo i operacions
7	CompteCorrent	Concreta	Compte amb targetes i productes associats
8	CompteTermini	Concreta	Compte amb durada fixada en mesos
9	TargetaCredit	Concreta	Targeta associada a comptes corrents
10	FonsInversio	Concreta	Producte d'inversió amb rendibilitat
11	CarteraValors	Concreta	Producte compost per valors borsaris

12	Valor	Concreta	Títol borsari dins una cartera
13	Banc	Concreta	Entitat principal del sistema

1.3 Atributs per classe

Classe	Atributs
Persona	dni, nom, adreca, telefon
Client	(hereta Persona) + comptes: List<Compte>
Empleat	(hereta Persona) + sucursal: Sucursal
Sucursal	identificador, adreca, empleats: List<Empleat>
Compte	numeroCompte, dataObertura, saldo, tipusInteres, clients: List<Client>
CompteCorrent	(hereta Compte) + targetes, fonsInversio, carteres
CompteTermini	(hereta Compte) + nombreMesos: int
TargetaCredit	tipus, numero, titular, dataCaducitat
FonsInversio	nom, importFons, rendibilitat, dataObertura, dataVenciment
CarteraValors	valors: List<Valor>
Valor	nom, nombreTitols, preuCotitzacio
Banc	nom, sucursals: List<Sucursal>, clients: List<Client>

1.4 Mètodes principals

Classe	Mètodes
Compte	ingressar(double), retirar(double), consultarSaldo()
CompteCorrent	afegirTargeta(), afegirFons(), afegirCartera()
CompteTermini	calcularInteressos()
CarteraValors	afegirValor(), eliminarValor(), valorTotal()

Banc	afegirSucursal(), cercarClient(), afegirClient()
Sucursal	afegirEmpleat(), eliminarEmpleat()
Client	afegirCompte(), eliminarCompte()

2. Justificació de Classes i Relacions

2.1 Relacions d'Herència

Relació	Justificació
Persona → Client, Empleat	Comparteixen DNI, nom, adreça, telèfon. Factorització en superclasse abstracta (DRY).
Producte → Compte, FonsInversio, CarteraValors	"Productes associats als clients" - generalització abstracta.
Compte → CompteCorrent, CompteTermini	"Hi ha dos tipus de comptes" - comparteixen atributs però difereixen en funcionalitat.

2.2 Relacions d'Agregació

Relació	Cardinalitat	Justificació
Banc ◇— Sucursal	1 a 1..*	El banc té sucursals, però existeixen independentment.
Sucursal ◇— Empleat	1 a 1..*	L'empleat pot ser traslladat a una altra sucursal.
CompteCorrent ◇— TargetaCredit	1 a 0..*	La targeta existeix independentment del compte.
CompteCorrent ◇— FonsInversio	1 a 0..*	Producte independent associat al compte.
CompteCorrent ◇— CarteraValors	1 a 0..*	Producte independent associat al compte.

2.3 Relació de Composició

Relació	Cardinalitat	Justificació
---------	--------------	--------------

CarteraValors ◆— Valor	1 a 1..*	"Estan compostes per valors" - cicle de vida dependent.
---------------------------	----------	--

2.4 Relació d'Associació

Relació	Cardinalitat	Justificació
Client — Compte	1..* a 1..*	Un client pot tenir múltiples comptes i un compte múltiples titulars.

2.5 Relacions que NO apareixen

- **Dependència:** No hi ha cap ús puntual/temporal d'una classe per una altra sense mantenir referència.
- **Interfície:** L'enunciat no descriu contractes abstractes que diferents jerarquies hagin de complir.

3. Codi Java - Classes del Model

S'han implementat 13 classes Java amb JavaDoc complet. A continuació es mostren les classes més representatives.

3.1 Persona.java (classe abstracta)

```
package model;

/**
 * Classe abstracta que representa una persona dins el sistema bancari.
 * Superclasse comuna de Client i Empleat.
 * @author Alumne
 * @version 1.0
 */
public abstract class Persona {
    private String dni;
    private String nom;
    private String adreca;
    private String telefon;

    public Persona(String dni, String nom, String adreca, String telefon) {
        this.dni = dni;
        this.nom = nom;
        this.adreca = adreca;
        this.telefon = telefon;
    }
}
```

```

    }
    // Getters i Setters per a tots els atributs...
}

```

3.2 Compte.java (classe abstracta)

```

package model;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public abstract class Compte extends Producte {
    private String numeroCompte;
    private LocalDate dataObertura;
    private double saldo;
    private double tipusInteres;
    private List<Client> clients;

    public Compte(String numeroCompte, LocalDate dataObertura,
        double saldo, double tipusInteres) {
        super();
        this.numeroCompte = numeroCompte;
        this.dataObertura = dataObertura;
        this.saldo = saldo;
        this.tipusInteres = tipusInteres;
        this.clients = new ArrayList<>();
    }

    public void ingressar(double quantitat) { }
    public void retirar(double quantitat) { }
    public double consultarSaldo() { return this.saldo; }
    public void afegirClient(Client client) { this.clients.add(client); }
    // Getters, Setters i resta de mètodes...
}

```

3.3 CompteCorrent.java

```

public class CompteCorrent extends Compte {
    private List<TargetaCredit> targetes;
    private List<FonsInversio> fonsInversio;
    private List<CarteraValors> carteres;

    public CompteCorrent(String num, LocalDate data,
        double saldo, double interes) {
        super(num, data, saldo, interes);
        this.targetes = new ArrayList<>();
        this.fonsInversio = new ArrayList<>();
        this.carteres = new ArrayList<>();
    }

    public void afegirTargeta(TargetaCredit t) { targetes.add(t); }
    public void afegirFons(FonsInversio f) { fonsInversio.add(f); }
    public void afegirCartera(CarteraValors c) { carteres.add(c); }
    // Eliminadors, Getters i Setters...
}

```


3.4 CarteraValors.java (Composició amb Valor)

```
public class CarteraValors extends Producte {
    private List<Valor> valors; // COMPOSICIÓ: valors depenen de la cartera

    public CarteraValors() {
        super();
        this.valors = new ArrayList<>();
    }

    public void afegirValor(Valor valor) { valors.add(valor); }
    public void eliminarValor(Valor valor) { valors.remove(valor); }
    public double valorTotal() { return 0.0; }
}
```

3.5 Banc.java (Entitat arrel)

```
public class Banc {
    private String nom;
    private List<Sucursal> sucursals; // AGREGACIÓ
    private List<Client> clients;

    public Banc(String nom) {
        this.nom = nom;
        this.sucursals = new ArrayList<>();
        this.clients = new ArrayList<>();
    }

    public void afegirSucursal(Sucursal s) { sucursals.add(s); }
    public Client cercarClient(String dni) { return null; }
    // Resta de mètodes...
}
```

3.6 Resum de totes les classes implementades

Classe	Tipus	Hereta de	Atributs	Mètodes clau
Persona	abstract	—	4	Getters/Setters
Client	concreta	Persona	1 (List)	afegirCompte, eliminarCompte
Empleat	concreta	Persona	1	Getters/Setters
Sucursal	concreta	—	3	afegirEmpleat, eliminarEmpleat
Producte	abstract	—	0	—

Compte	abstract	Producte	5	ingressar, retirar, consultarSaldo
CompteCorrent	concreta	Compte	3 (Lists)	afegirTargeta, afegirFons, afegirCartera
CompteTermini	concreta	Compte	1	calcularInteressos
TargetaCredit	concreta	—	4	Getters/Setters
FonsInversio	concreta	Producte	5	Getters/Setters
CarteraValors	concreta	Producte	1 (List)	afegirValor, eliminarValor, valorTotal
Valor	concreta	—	3	Getters/Setters
Banc	concreta	—	3	afegirSucursal, cercarClient

4. Diagrames UML

4.1 Diagrama de Classes

Diagrama complet amb totes les classes, atributs, mètodes i relacions amb cardinalitats.
Generat amb PlantUML i renderitzat a `plantuml.com` :

Diagrama de Classes - Gestio Bancaria

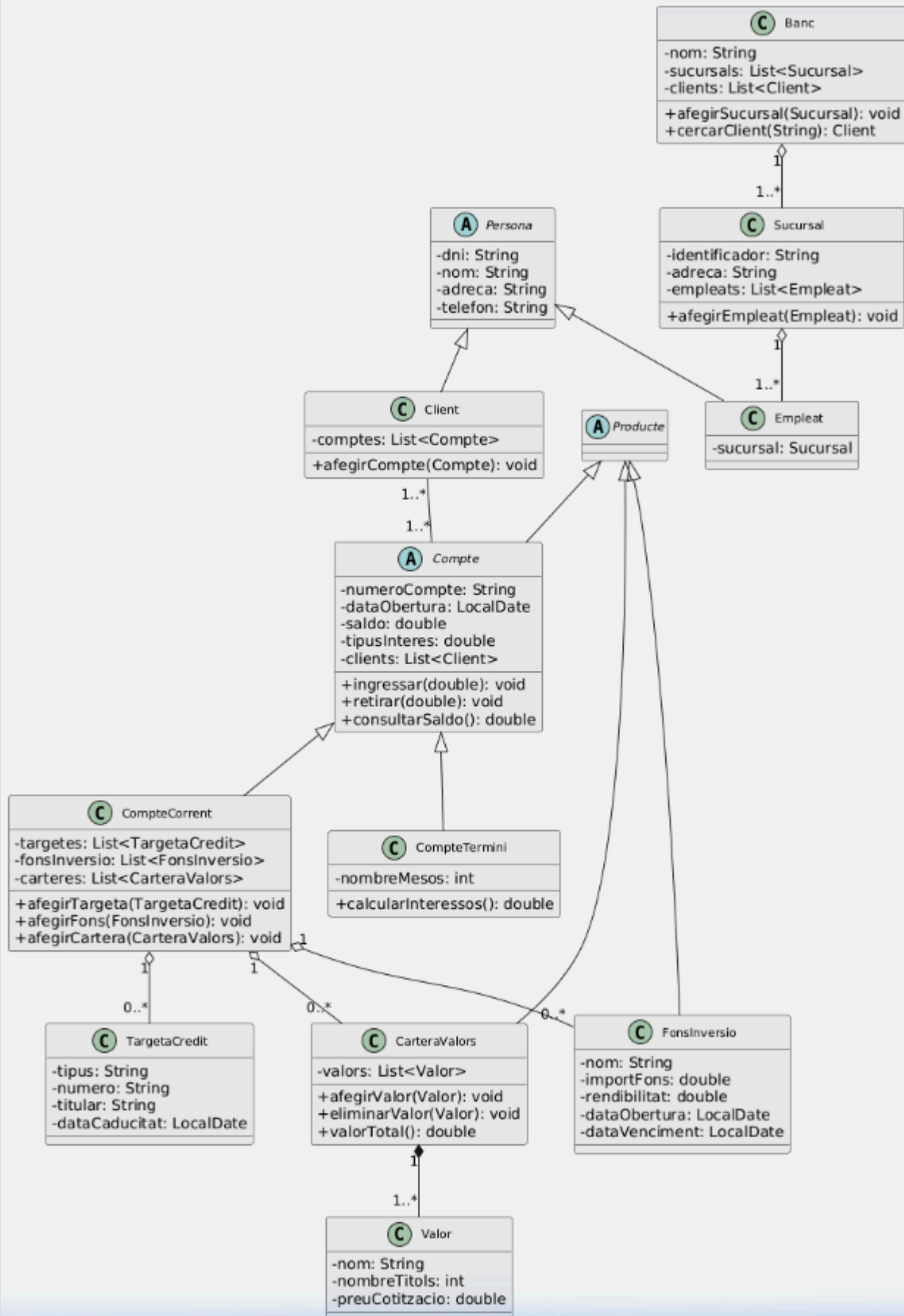


Figura 1: Diagrama de classes complet del sistema de gestió bancària.

4.2 Diagrama de Casos d'Ús

Actors: Client, Empleat, Administrador. Fitxer: `diagrames/casos-us.puml`

```
@startuml
actor "Client" as ActorClient
actor "Empleat" as ActorEmpleat
actor "Administrador" as ActorAdmin
rectangle "Sistema de Gestió Bancària" {
    usecase "Consultar saldo" as UC1
    usecase "Realitzar ingrés" as UC2
    usecase "Obrir compte" as UC6
    usecase "Associar targeta de crèdit" as UC8
    usecase "Contractar fons d'inversió" as UC9
    usecase "Comprar valors" as UC11
    usecase "Gestionar sucursals" as UC14
}
ActorClient --> UC1
ActorClient --> UC2
ActorEmpleat --> UC6
ActorEmpleat --> UC8
ActorEmpleat --> UC9
ActorEmpleat --> UC11
ActorAdmin --> UC14
@enduml
```

4.3 Diagrama de Seqüència

Cas: "Obrir un compte corrent amb targeta de crèdit". Fitxer: `diagrames/sequencia.puml`

```
@startuml
actor "Empleat" as emp
participant "Banc" as banc
participant "Client" as cli
participant "CompteCorrent" as cc
participant "TargetaCredit" as tc

emp -> banc : cercarClient(dni)
banc --> emp : client trobat/null
emp -> cc ** : new CompteCorrent(...)
emp -> cc : afegirClient(client)
emp -> cli : afegirCompte(compteCorrent)
emp -> tc ** : new TargetaCredit(...)
emp -> cc : afegirTargeta(targeta)
@enduml
```

4.4 Diagrama de Comunicació

Mateix escenari amb missatges numerats. Fitxer: `diagrames/comunicacio.puml`

4.5 Diagrama d'Activitat

Procés: "Realitzar transferència entre comptes" amb decisions (saldo suficient?). Fitxer:

```
diagrames/activitat.puml
```

4.6 Compilació satisfactòria

Totes les classes compilen correctament sense errors:

```
$ javac -d bin src/model/*.java
✓ Compilació exitosa - 0 errors
```

5. Patrons de Disseny

5.1 Què són els patrons de disseny?

Els **patrons de disseny** són solucions generals i reutilitzables a problemes comuns en el disseny de programari orientat a objectes. Van ser popularitzats pel llibre "*Design Patterns: Elements of Reusable Object-Oriented Software*" (1994), escrit pel **Gang of Four (GoF)**: Gamma, Helm, Johnson i Vlissides. Cataloguen 23 patrons en tres categories: **creacionals**, **estructurals** i **de comportament**.

5.2 Objectiu

- Proporcionar solucions reutilitzables a problemes recurrents
- Establir un vocabulari comú entre desenvolupadors
- Encapsular l'experiència de desenvolupadors experts
- Promoure dissenys flexibles i mantenibles

5.3 Avantatges i Inconvenients

Avantatges	Inconvenients
Reutilització de solucions provades	Complexitat afegida en projectes petits
Millor mantenibilitat del codi	Corba d'aprenentatge elevada

Comunicació eficient entre l'equip	Risc de sobreenginyeria
Bones pràctiques demostrades	Rigidesa prematura del disseny

5.4 Patrons més usats actualment

Patró	Categoria	Descripció	Exemple real
Singleton	Creacional	Una única instància global	Logger, pool de connexions BD
Factory Method	Creacional	Delega la creació a subclasses	Calendar.getInstance()
Abstract Factory	Creacional	Famílies d'objectes relacionats	Toolkits gràfics (Swing, GTK)
Builder	Creacional	Construcció d'objectes complexos	StringBuilder, HTTP Request
Observer	Comportament	Notificació automàtica de canvis	Event listeners en GUI
Strategy	Comportament	Algorismes intercanviables	Algorismes d'ordenació
Decorator	Estructural	Afegeix funcionalitat dinàmica	BufferedInputStream
Adapter	Estructural	Adapta interfícies incompatibles	Arrays.asList()
MVC	Arquitectònic	Separa Model-Vista-Controlador	Spring MVC, Django
Repository	Arquitectònic	Encapsula accés a dades	Spring Data JPA
Dependency Injection	Arquitectònic	Injecció externa de dependències	Spring Framework

5.5 Aplicació al projecte bancari

Patró	On s'aplicaria	Benefici
Singleton	Banc	Garantir una única instància del banc
Factory Method	Creació de Producte	Centralitzar la creació de productes
Strategy	Càlcul d'interessos	Intercanviar algorismes dinàmicament
Observer	Moviments de Compte	Notificar canvis automàticament

Exemple: Singleton per a Banc

```
public class Banc {
    private static Banc instancia;

    private Banc(String nom) {
        this.nom = nom;
        this.sucursals = new ArrayList<>();
    }

    public static Banc getInstance(String nom) {
        if (instancia == null) {
            instancia = new Banc(nom);
        }
        return instancia;
    }
}
```

Exemple: Strategy per a càlcul d'interessos

```
public interface EstrategiaInteres {
    double calcularInteres(double saldo, double interes, int meses);
}

public class InteresSimple implements EstrategiaInteres {
    public double calcularInteres(double saldo, double interes, int meses) {
        return saldo * (interes / 100) * (meses / 12.0);
    }
}
```


6. Documentació JavaDoc

S'ha generat la documentació JavaDoc completa en format HTML amb la comanda:

```
$ javadoc -d documentacio -sourcepath src -subpackages model \
  -encoding UTF-8 -charset UTF-8 -author -version -private
✓ Documentació generada a documentacio/index.html
```

Cada classe i mètode públic inclou:

- `@author` - Nom de l'autor
- `@version` - Versió 1.0
- `@param` - Descripció de cada paràmetre
- `@return` - Descripció del valor de retorn
- `@see` - Referències a classes relacionades

6.1 Exemple de JavaDoc complet (Classe Compte)

```
/**
 * Classe abstracta que representa un compte bancari genèric
 * dins el sistema de gestió bancària.
 * <p>
 * {@code Compte} és un {@link Producte} bancari que modela
 * els atributs i operacions comuns a tots els tipus de comptes.
 * </p>
 *
 * @author Alumne
 * @version 1.0
 * @see Producte
 * @see CompteCorrent
 * @see CompteTermini
 * @see Client
 */
public abstract class Compte extends Producte {

    /**
     * Realitza un ingrés al compte, augmentant el saldo.
     * @param quantitat la quantitat a ingressar (positiva)
     */
    public void ingressar(double quantitat) { }

    /**
     * Consulta i retorna el saldo actual del compte.
     * @return el saldo actual del compte en euros
     */
    public double consultarSaldo() {
        return this.saldo;
    }
}
```

La documentació completa en HTML es pot consultar obrint el fitxer `documentacio/index.html` al navegador.

7. Estructura del Projecte

```
gestio-bancaria/  
├── src/  
│   └── model/  
│       ├── Persona.java           ← Classe abstracta (superclasse)  
│       ├── Client.java           ← Hereta de Persona  
│       ├── Empleat.java          ← Hereta de Persona  
│       ├── Sucursal.java         ← Oficina bancària  
│       ├── Producte.java         ← Classe abstracta (productes)  
│       ├── Compte.java           ← Abstracta (hereta de Producte)  
│       ├── CompteCorrent.java    ← Hereta de Compte  
│       ├── CompteTermini.java    ← Hereta de Compte  
│       ├── TargetaCredit.java     ← Associada a CompteCorrent  
│       ├── FonsInversio.java     ← Hereta de Producte  
│       ├── CarteraValors.java     ← Hereta de Producte  
│       ├── Valor.java            ← Composició amb CarteraValors  
│       └── Banc.java             ← Entitat arrel  
├── diagrames/  
│   ├── diagrama-classes.puml     ← 5 diagrames UML  
│   ├── casos-us.puml  
│   ├── sequencia.puml  
│   ├── comunicacio.puml  
│   └── activitat.puml  
├── documentacio/                 ← JavaDoc HTML generat  
├── docs-practica/  
│   ├── analisi-gramatical.md  
│   ├── justificacio-classes.md  
│   └── patrons-disseny.md  
├── README.md  
└── generar-javadoc.sh
```

8. Conclusions

- S'han identificat **13 classes** mitjançant l'anàlisi gramatical (mètode Booch).
- S'han justificat totes les **relacions**: 3 herències, 5 agregacions, 1 composició i 1 associació.
- S'han implementat totes les classes en **Java** amb atributs, constructors, getters/setters i mètodes específics.
- S'ha generat **JavaDoc complet** amb @author, @version, @param, @return i @see.
- S'han creat **5 diagrames UML** en PlantUML: classes, casos d'ús, seqüència, comunicació i activitat.
- S'han analitzat els **patrons de disseny** i la seva aplicabilitat al projecte.